

What is Memory (RAM)?

Memory or **RAM (Random Access Memory)** is a temporary storage area used by the computer while a program is running.

☞ You can think of it as a large collection of **lockers**.

- Each locker has a unique number → called an **Address**
- Each locker stores a small piece of data → usually **1 byte**

📁 Example

Suppose the RAM looks like this:

```
0x100 → Empty
0x200 → Empty
0x300 → Empty
0x400 → Empty
0x500 → Empty
```

☞ Here:

- 0x100, 0x200, etc. are **memory addresses**
- Empty means no data is stored yet

How Programs Organize Memory

Your program's memory isn't just one big bucket. It is divided into zones.

1. Code Segment: Where your compiled instructions live (Read Only).

এখানে থাকে তোমার program এর **code (instructions)**

- তুমি যে code লিখো → compile হওয়ার পর এখানে থাকে
- এটা **read-only** (change করা যায় না)

Example:

```
printf("Hello");
```

 এই instruction এখানে থাকবে

2. Data Segment: Global variables (available everywhere). Example: `int x = 10;`

3. Stack: Temporary variables (Function calls, local variables). Example:

```
void func() {
    int a = 5; // stack এ থাকবে
}
```

function শেষ হলেই এই memory **automatic delete** হয়ে যায় **and very faster**

4. **Heap**: Long-term memory (Objects/Data structures). Like : যেমন: malloc, new

মূল ধারণা: Stack কি?

☞ Stack = function call গুলোর একটা list (LIFO system)

মানে: Last আসলে First বের হয়

Stack Frame কি?

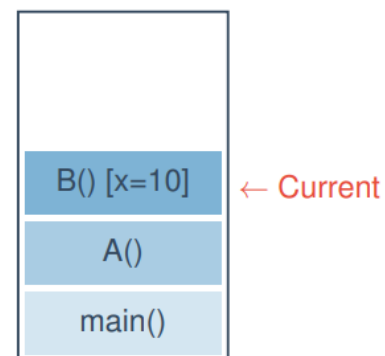
☞ যখন একটা function call হয়, তখন stack এ একটা **box (frame)** তৈরি হয়

Purpose: Keeps track of function calls.

- When you call a function, a "**Stack Frame**" is pushed.
- A Frame contains:
 - 1 Parameters (Arguments passed to function).
 - 2 Local Variables (Created inside function).
 - 3 Return Address (Where to go back to).
- When function ends, the Frame is **Popped** (deleted instantly).

Visualizing Stack Execution

```
void B() {  
    int x = 10;  
}  
  
void A() {  
    B();  
}  
  
int main() {  
    A();  
    return 0;  
}
```



Stack Properties

Allocation is Automatic: You don't ask for memory. You just declare int x;

Deallocation is Automatic: When the closing curly brace } is reached, memory is gone.

Scope Rule: You cannot access a variable in 'main()' from inside 'B()'. Why? Because they are in different stack frames (different "rooms").

1. Allocation is Automatic (নিজে নিজে memory নেয়)

☞ তুমি যখন লিখো:

```
int x = 5;
```

☞ তখন তুমি আলাদা করে memory চাইছো না

✗ তুমি বলো নাই: “আমাকে memory দাও”

✓ Stack নিজে নিজে x এর জন্য জায়গা বানিয়ে দেয়

2. Deallocation is Automatic (নিজে নিজে মুছে যায়)

☞ যখন function শেষ হয় {} এ পৌঁছায়

```
void B() {  
    int x = 10;  
} // এখানে x delete হয়ে যায়
```

☞ তখন x automatic delete হয়ে যায়

3. Scope Rule (কেন অন্য function থেকে variable দেখা যায় না?)

☞ Example:

```
void B() {  
    int x = 10;  
}  
  
int main() {  
    // এখানে x access করা যাবে না  
}
```

☞ কেন?

কারণ:

- x আছে শুধু B() এর ভিতরে
- main() আর B() আলাদা

🏠 Easy Analogy (সবচেয়ে important)

ভাবো:

- প্রতিটা function = আলাদা room 🏠
- Stack frame = ওই room

👉 B() এর room এ:

x = 10

👉 main() এর room এ:

x নাই ✕

👉 তাই main() গিয়ে B() এর x দেখতে পারে না

💧 এক লাইনে

- ✓ Stack নিজে memory দেয়
- ✓ নিজে remove করে
- ✓ এক function এর variable অন্য function দেখতে পারে না

How does GC know what is "Garbage"?

The definition of Garbage:

"An object that can no longer be reached by the running program."

" If you lose the "Remote Control" (Reference), you can never use the TV (Object) again. It becomes garbage."

Remote Control Example (সবচেয়ে সহজ)

ভাবো:

- 📺 TV = Object
- 🕒 Remote = Reference (pointer / variable)

👉 তুমি TV চালাতে পারো কারণ তোমার কাছে remote আছে

✗ যদি remote হারিয়ে যায়?

☞ তখন:

- TV আছে (memory তে object আছে)
- কিন্তু তুমি control করতে পারছো না

☞ এই TV এখন Garbage

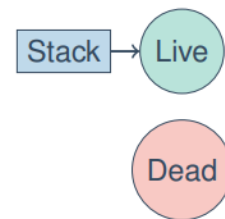
🔍 Mark and Sweep কি?

Algorithm 2: Mark and Sweep (Java)

Idea: Occasionally stop the world and trace everything.

Step 1: Mark Start from the Stack ("Roots").
Follow every pointer. Mark every object found as "Alive".

Step 2: Sweep Scan the Heap. Any object NOT marked is dead. Delete it.



☞ এটা একটা Garbage Collection Algorithm

মানে:

☞ কোন object দরকারি (live) আর কোনটা useless (garbage) — সেটা খুঁজে বের করে delete করে

💡 Main Idea

☞ মাঝে মাঝে program একটু pause করে

☞ তারপর দেখে: “কোন object এখনো use হচ্ছে?”

◆ Step 1: Mark (চিহ্ন দেওয়া)

☞ শুরু হয় Stack (root) থেকে

- Stack এ যেগুলো variable আছে → সেগুলো valid (live)
- সেগুলো যাদের দিকে point করে → সেগুলোও live

☞ এভাবে সব reachable object কে **mark (✓)** করা হয়

★ সহজভাবে:

যেগুলো পাওয়া যায় = LIVE (use হচ্ছে)

🔗 Example

Stack → A → B → C

☞ তাহলে:

- A, B, C → সব LIVE

◆ Step 2: Sweep (পরিষ্কার করা)

☞ এখন Heap check করা হয়

- যেগুলো mark করা হয়নি ✗
- মানে যেগুলোতে পৌঁছানো যায় না

☞ সেগুলো delete করে দেওয়া হয়

★ সহজভাবে:

যেগুলো পাওয়া যায় নাই = DEAD = DELETE ✗

□ Easy Analogy

ভাবো তুমি একটা room clean করছো 🏠

Step 1 (Mark):

☞ যেগুলো দরকারি জিনিস → mark করো ✓

Step 2 (Sweep):

☞ যেগুলো mark নাই → ফেলে দাও 🗑️

The Cost of Magic

GC makes life easy, but it is not free.

Performance Overhead: The CPU has to spend time counting/sweeping instead of running your app.

Unpredictable Pauses: Sometimes the program "freezes" for 10ms while GC runs.

Memory Footprint: GC languages often use more RAM than C++.

চল একদম সহজভাবে বুঝি ☺

☐ “The Cost of Magic” (GC এর সমস্যা)

☞ Garbage Collection (GC) life easy করে ✓

☞ কিন্তু এটা free না ✗ → কিছু downside আছে

◆ 1. Performance Overhead (Extra কাজ)

☞ GC চালাতে CPU কে আলাদা কাজ করতে হয়

- Garbage খুঁজে বের করা
- Mark & Sweep করা

☞ এই সময় CPU তোমার program এর কাজ কম করে

★ সহজভাবে:

Program একটু slow হয়ে যায় কারণ GC extra কাজ করে

◆ 2. Unpredictable Pauses (হঠাৎ freeze)

☞ GC চলার সময় program হঠাৎ থেমে যেতে পারে ☹

- ধরো 10ms এর জন্য pause হলো
- user বুঝতে পারে lag হচ্ছে

★ Example:

- game খেলছো 🎮 → হঠাৎ lag
- app use করছো → একটু freeze

★ সহজভাবে:

GC মাঝে মাঝে program pause করে দেয় (unexpected)

◆ 3. Memory Footprint (বেশি RAM লাগে)

☞ GC language (যেমন Java, Python) usually বেশি memory use করে

☞ কারণ:

- garbage সাথে সাথে delete হয় না
- কিছু সময় memory তে পড়ে থাকে

☞ C++ এ:

- তুমি নিজে delete করো → কম memory লাগে

★ সহজভাবে:

GC = বেশি RAM খায়

🔥 এক লাইনে

☞ GC easy করে, কিন্তু slow + pause + বেশি memory লাগে

Class Quiz 1: Where does it go?

I will show a line of code. You say: STACK or HEAP.

1. `int age = 25;`
2. `String s = new String("Hello");`
3. `double[] prices = new double[100];`
4. The variable prices itself?

Code	STACK	HEAP
<code>int age = 25</code>	age	—
<code>String s = new String("Hello")</code>	s	"Hello"
<code>double[] prices = new double[100]</code>	prices	array
<code>prices itself</code>	Prices..	—

এক লাইনে

☞ Name (variable) → STACK

☞ Data (object/array) → HEAP

```
int * getNumber () {
    int x = 50;
    return &x ; // Returning address of x
}

void main () {
    int * p = getNumber () ;
    cout << * p ;
}
```

Bug কি?

x হলো local variable।

- x STACK-এ তৈরি হয় যখন `getNumber()` ফাংশন চালানো হয়।
- ফাংশন শেষ হলে x মুছে যায়।
- তাই `return &x;` মানে তুমি STACK-এর একটি মুছে যাওয়া ঠিকানা ফেরত দিচ্ছ।
- এর ফলে *p ব্যবহার করলে **undefined behavior** হবে।

Fix করার উপায়:

1 HEAP ব্যবহার করা (dynamic allocation)

```
int * getNumber() {
    int* x = new int(50); // heap-এ allocate
    return x;
}

int main() {
    int* p = getNumber();
    cout << *p;
    delete p; // memory clean-up
}
```